

Sistema de recuperación de información sobre fuentes abiertas

CODEFEST AD ASTRA 2026 — Etapa 1 · Equipo Talon Systems

Daniel Benavides · Samuel Rozen · Camilo Molina · Andrés Felipe Alfonso
Universidad de los Andes · Fuerza Aeroespacial Colombiana

1. Resumen

Sistema de recuperación que, dada una pregunta en lenguaje natural, devuelve los diez fragmentos de texto y los tres documentos más relevantes de un corpus de 1.826 archivos en español, inglés y portugués. No genera texto: busca y ordena.

El pipeline recorre siete fases —extracción, limpieza, fragmentación, codificación, indexación, recuperación y evaluación — más un grafo de conocimiento opcional. Toda la recuperación se apoya en modelos *encoder-only*, sin ninguna arquitectura decoder en ninguna etapa (§8.3).

Las decisiones que más afectan al resultado, y que este informe justifica, son tres: fragmentar fino e **expandir en la salida**, elegir el encoder por coste de cómputo antes que por un benchmark sin resolución estadística, y tratar la robustez de la ingesta como un problema de puntaje y no de higiene.

2. El corpus

El índice provisto por ADL describe 1.826 documentos repartidos así:

Fenómeno	JSON	PDF	PBF	CSV	Imagen	XLSX	TXT	Total
1 · IA y capacidades estratégicas	205	231	0	19	0	4	0	459
2 · Seguridad espacial	230	237	0	2	8	0	1	479
3 · Dinámicas territoriales	519	291	73	5	0	0	0	888
Total	954	759	73	26	8	4	1	1.826

Dos observaciones condicionaron el diseño.

No hay ningún archivo HTML, pese a que §1.3 lo lista. El extractor correspondiente se implementó igualmente, porque el coste es bajo y el riesgo de una ampliación del corpus no lo es.

El grueso del corpus es JSON, no PDF. Los 954 JSON provienen de 16 observatorios con esquemas distintos. Escribir un adaptador por fuente no escalaba, así que el extractor aprovecha lo que casi todos comparten: una o varias claves de contenido con prosa (`content`, `body`, `resumen`, `articleBody`) y el resto metadatos. El cuerpo se emite tal cual —es texto natural y el encoder lo aprovecha mejor sin prefijos— y los metadatos como `clave: valor`, que es lo que les da sentido recuperable: `2024` no dice nada, `año: 2024` sí.

Riesgo abierto. 59 nombres de archivo se repiten en carpetas distintas, afectando a 186 documentos (por ejemplo, el mismo informe del CSET en `pdfs/Reports` y en `pdfs/Translation`). Como el emparejamiento del ground truth a nivel documento se hace por `fuelle` (§10.2.1), esos 186 documentos son ambiguos si `fuelle` es el nombre a secas. Se consultó a la organización; la metadata conserva `fuelle` y la ruta relativa para poder cambiar de criterio sin reindexar.

3. Fase 1 — Extracción

Un extractor por formato, tras un punto de entrada único `extraer(path, formato)`.

Formato	Herramienta	Nota
PDF	PyMuPDF + Tesseract	bloques ordenados por posición; OCR por página
JSON	<code>json</code> + aplanado por rol de clave	separa cuerpo de metadatos
CSV / XLSX	<code>pandas</code> / <code>openpyxl</code>	encabezados repetidos por fila
Imagen	Tesseract (opcional EasyOCR)	OCR más el nombre del archivo
PBF	<code>osmium</code>	topónimos agrupados por tipo
HTML	BeautifulSoup	rama preventiva

PDF. Se extrae con `get_text("blocks", sort=True)`, que ordena por posición y respeta el doble columnado; la extracción lineal entrelaza las dos columnas y produce frases sin sentido que el encoder no puede aprovechar. El OCR se decide **por página y no por documento**, de modo que un PDF mixto se recupera entero y solo se paga OCR donde falta capa de texto.

El pack de idioma del OCR no es un detalle. Medido sobre una portada en portugués: con el pack `eng` el CER es

0,036 y `Relatório` sale como `Relatério`; con `por`, 0,000. La corrupción es silenciosa —no hay error, solo diacríticos mal— y degrada a la vez el embedding y la detección de idioma. Combinar packs (`spa+por+eng`) resultó peor (0,012) que acertar con uno solo. Para un PDF escaneado entero, donde no hay texto previo del que inferir el idioma, se hace una primera pasada con el pack combinado, se detecta el idioma sobre ese texto sucio y se repite el OCR con el correcto.

Robustez frente a archivos corruptos. Se evaluó `unstructured` como alternativa. Descartada por dos motivos: su modo `hi_res` —el único que clasifica encabezados y pies— descarga modelos de layout en tiempo de ejecución, lo que rompe la reproducibilidad exigida en §1.4; y es más frágil ante archivos mal formados. El caso que zanjó la decisión: uno de los archivos entregados con extensión `.pdf` es en realidad un ZIP con imágenes JPEG dentro. `unstructured` falla con `PDFPageCountError`; PyMuPDF lo abre y recupera 7.243 palabras de sus 24 páginas. Con 1.826 documentos de 20 observatorios, un documento perdido es F1@3 perdido.

Se descartó también `marker-pdf`, pese a su mejor fidelidad de layout: sus pesos usan una licencia con restricción comercial incompatible con el requisito de licencia libre, descarga ~1 GB en la primera ejecución, y optimiza una fidelidad tipográfica que la métrica no puntúa.

Fallo ruidoso. Un documento que no produce texto lanza `DocumentoSinTexto` en vez de devolver vacío. Un documento con cero fragmentos no está en el índice y es irrecuperable: si el ground truth lo marca relevante, ese F1@3 se pierde y nada en el pipeline lo delataría. Los fallos se acumulan en un CSV y la ingesta continúa.

4. Fase 2 — Limpieza y normalización

Normalización a NFC, resolución de ligaduras y unión de los guiones de corte de línea (`infraes-\ntructura` → `infraestructura`, que de otro modo el tokenizador parte en dos).

Eliminación de encabezados y pies por posición vertical. Con 759 PDFs institucionales, una cabecera repetida que sobreviva a la fragmentación hace que decenas de chunks empiecen igual y compitan entre sí en el ranking en vez de competir por su contenido.

Dos implementaciones previas basadas en el *orden* de línea dentro de la página fueron descartadas porque borraban párrafos: la última línea del cuerpo es tan «de borde» como el pie. Al medir las coordenadas reales de un PDF institucional la separación resultó nítida: cabecera en $y = 0,04\text{--}0,05$ de la altura de página, pie en $0,89\text{--}0,92$, cuerpo entre $0,12$ y $0,40$.

La versión final elimina una línea solo si cumple **tres** condiciones a la vez: está en la zona de cabecera o de pie, es corta (≤ 12 palabras) y se repite en al menos el 60 % de las páginas. Además incorpora tres salvaguardas: devuelve un informe auditable de qué eliminó; se abstiene en páginas OCR, que no tienen coordenadas fiables; y se abstiene por completo si fuese a borrar más del 25 % del documento. Sobre un PDF de 12 páginas elimina las 4 líneas de boilerplate (-16 % de texto) sin perder contenido.

Detección de idioma con `py3langid` restringido a es/pt/en. La restricción evita que un texto gallego o catalán se clasifique como tal y rompa el mapeo de reglas de segmentación. Medido sobre 1.200 muestras de Universal Dependencies por longitud: a 15 palabras o más el acierto es del 100 %; por debajo de 8 cae, y por eso el idioma se detecta **una vez por documento** sobre el texto completo y se propaga a todos sus fragmentos.

5. Fase 3 — Fragmentación

La oración es la unidad atómica y un chunk es un rango de oraciones. Con ese diseño, la completitud lingüística que exige §3.3 es una propiedad estructural: no puede salir un fragmento cortado, no hay que verificarlo después.

La segmentación usa `pysbd`. Como no soporta portugués, se mapea a las reglas de español mediante una tabla explícita; un `except` silencioso habría segmentado un tercio del corpus con otras reglas sin que nadie se enterara. La elección se verificó por F1 de fronteras contra tres treebanks de Universal Dependencies (es/pt/en), comparando cinco segmentadores. Las diferencias resultaron pequeñas (0,879 para `pysbd` frente a 0,894 para NLTK punkt) y el criterio decisivo fue otro: el diseño de dos niveles absorbe los errores de frontera, porque la salida se reconstruye expandiendo. `syntok` quedó descartado (0,687).

Arquitectura de dos niveles.

1. **Indexar fino:** chunks de ~140 palabras, **sin solape**. Chunks pequeños dan mayor precisión de recuperación y el embedding no se diluye.
2. **Expandir en la salida:** el fragmento entregado crece con las oraciones de sus vecinos inmediatos del mismo documento hasta ~235 palabras, con margen sobre el límite duro de 250.

El solape se eliminó a propósito: como la salida se expande con los vecinos, un pasaje partido por una frontera se recupera por cualquiera de sus mitades y se reconstruye al generar. El solape solo duplicaría texto entre chunks contiguos.

La expansión opera **oración a oración y no chunk a chunk**, porque un vecino entero casi nunca cabe ($140 + 140 > 235$). Medido sobre un índice real: los fragmentos entregados pasan de 138 a 230 palabras de media, **un 67 % más de evidencia** por fragmento, sin superar el límite. Como §10.2.1 establece que la relevancia se juzga sobre el contenido textual del campo `text` entregado, llenar el presupuesto aumenta directamente la probabilidad de cubrir el pasaje que el anotador marcó como relevante. Es la mejora de mayor retorno del sistema y no requiere tocar el modelo.

Se garantiza además que ninguna «oración» anómala desborde el límite: las filas de CSV serializadas y las tablas mal extraídas de PDF pueden superar por sí solas las 250 palabras, y se trocean por punto y coma, luego por coma y, como último recurso, por ventana de palabras, conservando siempre los separadores para no alterar el campo `texto`.

6. Fases 4 y 5 — Codificación e indexación

Encoder: `ibm-granite/granite-embedding-97m-multilingual-r2` (ModernBERT, 384 dimensiones, encoder-only, licencia Apache 2.0).

Conviene ser explícito sobre la fuerza de esta evidencia. El benchmark interno comparó tres candidatos sobre **tres consultas**, y los tres alcanzaron MRR 1,0 y `acierto@1` de 3/3 — incluida la prueba de prefijos de E5, que no mostró diferencia. Eso no demuestra que los prefijos sean irrelevantes; demuestra que con tres consultas no hay resolución estadística para distinguir nada. **El criterio real de la decisión fue el coste de cómputo:** 4,63 h para codificar el corpus completo frente a 23,85 h de granite-311m, sobre CPU. Con el plazo disponible, un encoder que no permite reindexar es un encoder que impide iterar.

Se verificó que ninguno de los candidatos fuese una arquitectura decoder. Esta comprobación no es trivial: varios de los mejores modelos de embedding del ranking MTEB (Qwen3-Embedding, gte-Qwen2, e5-mistral-7b) son decoders y quedan prohibidos por §8.3 aunque se usen solo para embeddings.

Índice: `IndexFlatIP` con **vectores normalizados a norma unitaria**, lo que equivale exactamente a similitud coseno. Es búsqueda exacta, sin la pérdida de recall de un índice aproximado. Con 50 consultas la latencia es irrelevante, así que no hay ninguna razón para sacrificar exactitud por velocidad.

La normalización vive en un solo sitio. `encoding.py` normaliza; `indexing.py` solo verifica y falla con `ValueError` si las normas no son 1. Repartir la regla entre dos módulos permitiría que el vector de consulta y los del índice se normalizaran con criterios distintos, y la búsqueda se degradaría sin lanzar ningún error.

Correspondencia FAISS ↔ metadata. La línea i de `metadata.jsonl` es el vector i del índice. Se verifica explícitamente tras `index.add()`, porque filtrar chunks después de indexar rompe esa correspondencia en silencio y hace que cada búsqueda devuelva el texto de otro fragmento.

7. Fase 6 — Recuperación y agregación

Para cada consulta: se codifica con el mismo encoder y los mismos prefijos que los pasajes, se recuperan `k = 50` fragmentos, se agregan a documentos y se entregan los 10 mejores fragmentos y los 3 mejores documentos.

Agregación por `max pooling`: un documento vale lo que vale su mejor fragmento. Es la primera opción que nombra §8.6 y la más estable de las tres. `suma` premia a los documentos largos, que aportan más fragmentos al top-k aunque ninguno sea bueno; `media` castiga al documento que aportó un fragmento excelente y varios mediocres. El desempate es explícito —puntuación, luego mejor posición alcanzada, luego `doc_id` alfabético— para que dos corridas idénticas no devuelvan documentos distintos cuando las puntuaciones empatan.

Ampliación automática de `k`. La entrega exige exactamente tres documentos por consulta. Si con `k = 50` no aparecen tres documentos distintos —posible cuando el top-k se concentra en uno o dos—, la búsqueda se reintenta con una `k` mayor hasta agotar el índice. Sin esto, una sola consulta pobre abortaría la generación completa y dejaría al equipo sin `resultados.jsonl`.

Tensión conocida. NDCG@10 y F1@3 tiran en direcciones opuestas: el top-10 de fragmentos tiende a concentrarse en uno o dos documentos, y con `max` puro el tercer documento sale de ruido. Como el ranking final se decide por conteo de Borda, que premia consistencia en ambas tablas antes que excelencia en una, se optó por no sacrificar F1@3 para exprimir NDCG@10.

8. Fase 7 — Evaluación

Se implementaron las dos métricas con las fórmulas literales de §10.2, no con las estándar. La diferencia importa: el denominador del recall es `min(|D*|, 3)`, no `|D*|`. Con cinco documentos relevantes y dos aciertos, la spec da $R = 2/3$ y el recall habitual daría $2/5$; el F1 pasa de 0,667 a 0,500. Cualquier implementación tomada de una librería devuelve el número equivocado.

Un segundo detalle resultó más peligroso todavía. El IDCG debe calcularse sobre las relevancias que el **ground truth** define para la consulta, no sobre las del resultado devuelto. Calculado sobre lo devuelto —el error natural—, un sistema que recupera 1 de 5 fragmentos relevantes y lo coloca primero puntúa 1,000, exactamente igual que uno que los recupera los cinco:

Fragmentos relevantes recuperados	IDCG sobre lo devuelto	IDCG sobre el ground truth
1 de 5	1,000	0,339
3 de 5	1,000	0,723
5 de 5	1,000	1,000

Con la primera columna, ninguna mejora de cobertura es medible y toda la rejilla de experimentos compararía ruido. La implementación se corrigió y quedó cubierta por 27 pruebas con valores calculados a mano.

El módulo incluye además un validador del esquema de `resultados.jsonl` que recorre las 50 líneas completas —no se detiene en el primer error— y verifica conteo, orden `q001→q050`, rangos consecutivos, presencia de campos y el límite de 250 palabras.

9. Grafo de conocimiento (bonus)

Entidades extraídas con `Babelscape/wikineural-multilingual-ner`, un modelo BERT multilingüe y por tanto compatible con §8.3. Las relaciones se establecen por co-ocurrencia dentro de un mismo fragmento, sin un modelo de extracción de relaciones dedicado: lo que la spec puntúa es que cada tripleta sea trazable a su fragmento de origen, y dos entidades que comparten un chunk de 140 palabras están relacionadas con alta probabilidad.

Cada nodo guarda tipo, número de apariciones y documentos de origen; cada arista guarda su peso y los `chunk_id` que la soportan. Esa trazabilidad es lo que habilita el uso opcional en recuperación descrito en §8.5. Se exporta a `grafo/grafo.graphml`.

10. Reproducibilidad

`generador.py` localiza el paquete `src/` tanto junto a sí mismo como un nivel por encima, de modo que el directorio de entrega funciona de forma autónoma y también desde el repositorio. `scripts/empaquetar.py` arma la entrega, comprueba que estén los cuatro entregables y la ejecuta desde una copia en un directorio temporal limpio, con `PYTHONPATH` vacío y otro directorio de trabajo, que es la única forma de detectar una dependencia del entorno del desarrollador.

Las dependencias van con versiones fijas. `pysbd` y `py3langid` en particular no tienen ruta de respaldo: si faltan, el proceso se detiene, porque un cambio en las reglas de segmentación cambia las fronteras de los chunks y, con ellas, `posicion` y `chunk_id`, que dejarían de corresponder a los vectores del índice sin lanzar ningún error.

La suite cubre 78 pruebas sobre fragmentación, limpieza, evaluación, codificación, indexación, recuperación, agregación y salida.

11. Limitaciones y trabajo futuro

El conjunto de evaluación propio es pequeño. El benchmark de encoders se apoyó en tres consultas, insuficiente para discriminar calidad. La ampliación a 40–60 consultas etiquetadas a mano, equilibradas entre los tres fenómenos y los tres idiomas, es el requisito previo para que la rejilla de experimentos —tamaño de chunk, estrategia de agregación, encoder— produzca conclusiones y no ruido.

No se implementó recuperación híbrida. BM25 fusionado con el índice denso vía Reciprocal Rank Fusion está permitido (no es un modelo generativo) y suele aportar en consultas con terminología específica. Quedó fuera por tiempo.

El reranking con cross-encoder queda pendiente de confirmación. Un cross-encoder encoder-only como `bge-reranker-v2-m3` (XLM-RoBERTa, Apache 2.0) no es un LLM y por lectura literal de §8.3 sería admisible, con una ganancia esperable de varios puntos de NDCG@10. Se implementaría tras respuesta escrita de la organización.

La ambigüedad de `fFuente` descrita en §2 sigue abierta y afecta a 186 documentos.